COMP – 304
Summer 2026

Advanced Android App Architecture and Development Practices

Objectives:

- **Discuss the fundamental principles of app architecture and its importance in Android development.**
- **Implement the Model-View-ViewModel (MVVM) architecture to enhance code separation and maintainability.**
- **Explain the functionality and benefits of LazyColumn in Jetpack Compose.**
- **Create composable functions using LazyColumn to efficiently display lists.**
- **Utilize Jetpack libraries to streamline and enhance app development.**
- **Apply dependency injection to improve code modularity and testability.**
- **Migrate an Android project to Kotlin Gradle DSL and manage dependencies using version catalogs**

Introduction to App Architecture

- Structured architecture in app development is important:
  - for scalability and maintainability.
  - enables easy handover to other developers or teams.

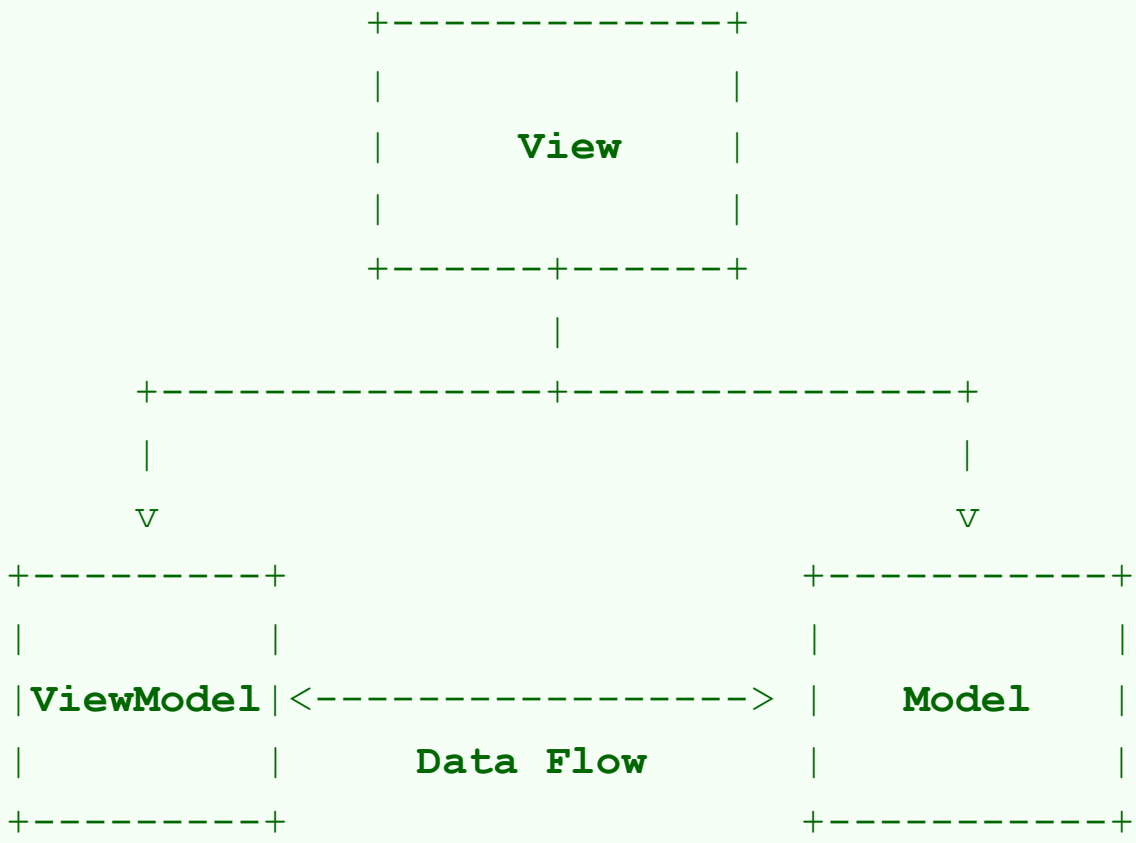
Understanding App Architecture

- App architecture refers to the framework that **organizes code in a way that supports scalability, maintenance, and development efficiency**.
- Benefits:
  - **Separation of concerns** reduces complexity.
  - Simplifies **testing and maintenance**.
  - Facilitates **easier scaling and team collaboration**.

Architectural Choices

- Overview of Architectures:
  - **MVVM**: Recommended by Google, separates data model layer from the user interface via the ViewModel.
  - **MVC**: Model controls business logic, View handles display, Controller manages communication.
  - **MVP**: Similar to MVC but with a Presenter facilitating the communication between View and Model.
  - **MVI**: Emphasizes a unidirectional data flow from a user's Intent to Model and View.

MVVM Architecture Overview



MVVM Architecture Overview

- **Components**:
  - **Model**: Manages the data and business logic.
  - **View**: Handles the layout and display.
  - **ViewModel**: Acts as a bridge between the Model and the View.
- **Benefits**: Encourages modular code, simplifies testing by isolating components.

Implementing MVVM - Model Layer

```
data class Pet(val id: Int, val name: String, val species: String)

interface PetsRepository {
    fun getPets(): List<Pet>
}

class PetsRepositoryImpl : PetsRepository {
    override fun getPets() = listOf(
        Pet(1, "Bella", "Dog"), Pet(2, "Luna", "Cat") // Add more pets
    )
}
```

- Explanation: Defines the **data structure and a repository for managing pet data**.

MVVM - ViewModel Layer

```
class PetsViewModel(private val petsRepository: PetsRepository) :
ViewModel() {
    fun getPets() = petsRepository.getPets()
}
```

- Explanation: **ViewModel retrieves data through the repository, providing an abstraction layer to the UI**.

MVVM - View Layer with Jetpack Compose

```
@Composable
fun PetsList(petsViewModel: PetsViewModel) {
    val pets = petsViewModel.getPets()
    LazyColumn {
        items(pets) { pet ->
            Text("Name: ${pet.name}, Species: ${pet.species}")
        }
    }
}
```

Explanation: Uses Jetpack Compose to render a list of pets in a scrollable column.

Deep Dive into Jetpack Libraries

- Key Libraries:
  - **Room**: Simplifies database access.
  - **LiveData**: Manages data observability.
  - **Navigation**: Facilitates screen transitions.
- Benefits: Reduces boilerplate, enhances app performance and maintenance.

Dependency Injection with Koin

- Benefits: Simplifies dependency management by decoupling object creation.

```
val appModules = module {
    single { PetsRepositoryImpl() as PetsRepository }
    viewModel { PetsViewModel(get()) }
}
```

Implementing Dependency Injection

```
class PetsViewModel(private val petsRepository:
PetsRepository) : ViewModel() {
    fun getPets() = petsRepository.getPets()
}
```

- Explanation: Injects PetsRepository into PetsViewModel, demonstrating dependency injection in practice.

Kotlin Gradle DSL Overview

- Benefits:
  - Code autocompletion and type safety.
  - Compile-time error detection improves build reliability.

Migrating to Kotlin Gradle DSL

- Migration Steps:
  - Rename .gradle files to .kts.
  - Update syntax to Kotlin, e.g., minSdk(24) to minSdk = 24.

Using Version Catalogs in Gradle

- Benefits:
  - Central management of dependencies.
  - Simplifies updates and consistency across modules.

Configuring Version Catalogs

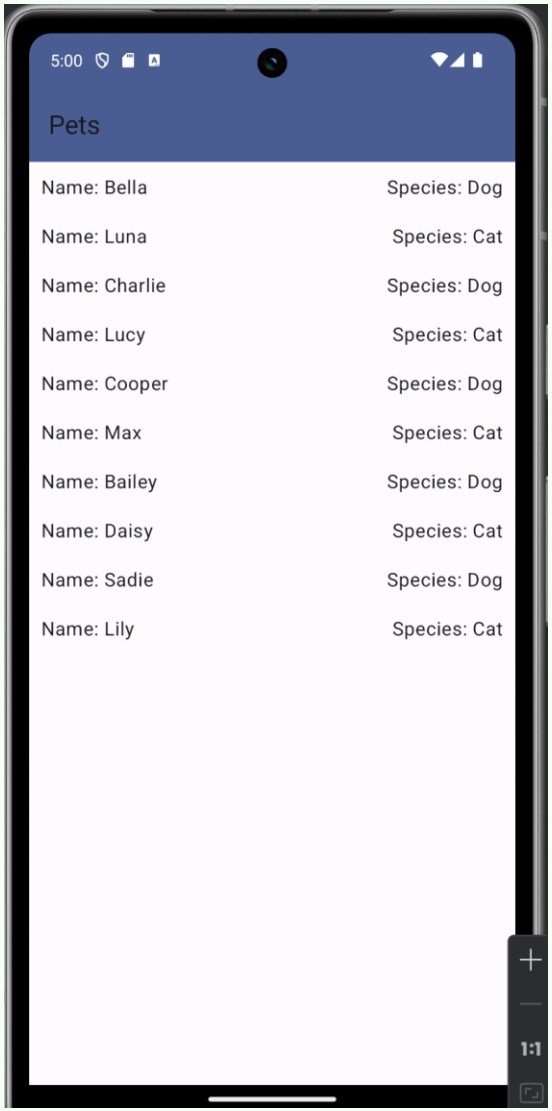
```
dependencies {
    implementation(libs.core.ktx)
    implementation(libs.compose.ui)
}
```

- Explanation: Shows how to use version catalogs to reference dependencies in build.gradle.kts.

Summary of Architectural Implementation

- Recap: Covered
  - MVVM architecture
  - Jetpack libraries
  - dependency injection
  - Kotlin Gradle DSL
  - version catalogs.

AppArchitecture Example



References

- Textbook Chapter 5
- https://developer.android.com/topic/architecture